

판다스(Pandas) DataFrame

생성 & 인덱싱 / 슬라이싱 강의 교재

광명코딩학원 | 파이썬 판다스 기초 강의

[핵심 개념] DataFrame은 2차원 데이터를 다루는 판다스의 자료구조입니다. 데이터(data) + 인덱스(index) + 컬럼(columns)으로 구성되며, 내부적으로 행번호와 열번호가 존재합니다.

PART 1. DataFrame 생성

1. DataFrame 내부 구조

열번호 0 1 2

행번호 증가 PER PBR

0 NAVER 157000 39.88 4.38

1 삼성전자 51300 8.52 1.45

2 LG전자 68800 10.03 0.87

3 카카오 140000 228.38 2.16

↑ ↑

인덱스 데이터 영역 컬럼(columns)

구성 요소	위치	설명
index	왼쪽 열	행 레이블 (NAVER, 삼성전자 등)
columns	상단 행	열 레이블 (증가, PER, PBR)
data	가운데	실제 데이터 영역 (2차원)
행번호	숨김	내부적으로 존재 — iloc로 접근
열번호	숨김	내부적으로 존재 — iloc로 접근

2. DataFrame 생성 — 방법 1: 컬럼 단위 (딕셔너리)

[개념] 컬럼명을 딕셔너리의 key로, 데이터를 values(리스트)로 표현합니다. columns 파라미터가 필요 없고 key가 자동으로 컬럼명이 됩니다.

방법 1 — 컬럼 단위

```
from pandas import DataFrame

data = {

    "종가": [157000, 51300, 68800, 140000],

    "PER": [39.88, 8.52, 10.03, 228.38],

    "PBR": [4.38, 1.45, 0.87, 2.16]

}

index = ['NAVER', '삼성전자', 'LG전자', '카카오']

df = DataFrame(data, index)

print(df)

# 종가 PER PBR

# NAVER 157000 39.88 4.38

# 삼성전자 51300 8.52 1.45

# LG전자 68800 10.03 0.87

# 카카오 140000 228.38 2.16
```

- 딕셔너리의 key → 컬럼명 자동 인식
- columns 파라미터 불필요
- 데이터가 컬럼 방향(세로)으로 들어올 때 적합

3. DataFrame 생성 — 방법 2: 로우 단위 (2차원 리스트)

[개념] 각 행을 리스트로 표현하여 2차원 리스트로 구성합니다. columns 파라미터를 반드시 넘겨야 합니다.

방법 2 — 로우 단위 리스트

```
from pandas import DataFrame

data = [

[157000, 39.88, 4.38], # NAVER 행

[51300, 8.52, 1.45], # 삼성전자 행

[68800, 10.03, 0.87], # LG전자 행

[140000, 228.38, 2.16] # 카카오 행

]

index = ['NAVER', '삼성전자', 'LG전자', '카카오']

columns = ['종가', 'PER', 'PBR']

df = DataFrame(data=data, index=index, columns=columns)

print(df)
```

- 2차원 리스트(이중 리스트)로 데이터 표현
- columns 파라미터 반드시 필요 ← 방법1과 차이!
- API 등에서 행 단위로 데이터가 들어올 때 append() 방식으로 축적 후 생성

4. DataFrame 생성 — 방법 3: 로우 단위 (딕셔너리 리스트)

[개념] 각 행을 딕셔너리로 표현합니다. key가 자동으로 컬럼명이 되어 columns 파라미터가 필요 없습니다.

방법 3 — 로우 단위 딕셔너리

```
from pandas import DataFrame

data = [

{"종가": 157000, "PER": 39.88, "PBR": 4.38},

{"종가": 51300, "PER": 8.52, "PBR": 1.45},
```

```

{"종가": 68800, "PER": 10.03, "PBR": 0.87},

{"종가": 140000, "PER": 228.38, "PBR": 2.16}

]

index = ['NAVER', '삼성전자', 'LG전자', '카카오']

df = DataFrame(data=data, index=index)

print(df)

```

- 딕셔너리의 key → 컬럼명 자동 인식
- columns 파라미터 불필요
- 방법2와 달리 각 행에 컬럼명(레이블)이 함께 포함됨

5. 3가지 방법 비교

방법	데이터 방향	데이터 형태	columns 필요
방법 1	컬럼(세로) 단위	{컬럼명: [값...]}	X — key가 컬럼명
방법 2	로우(가로) 단위	[[값, 값], [값, 값], ...]	O — 반드시 필요
방법 3	로우(가로) 단위	[{컬럼명:값, ...}, ...]	X — key가 컬럼명

PART 2. DataFrame 인덱싱 & 슬라이싱

6. 컬럼 선택 (단일 컬럼)

[개념] 대괄호['컬럼명']을 통해 단일 컬럼을 선택합니다. 결과는 1차원이므로 Series 타입으로 리턴됩니다.

단일 컬럼 선택

```
# 단일 컬럼 선택 -> Series 리턴
```

```
print(df['종가'])
```

```
# NAVER 157000
```

```
# 삼성전자 51300
```

```
# LG전자 68800
```

```
# 카카오 140000
```

```
# Name: 종가, dtype: int64
```

```
print(df['PER'])
```

```
# NAVER 39.88
```

```
# 삼성전자 8.52
```

```
# ...
```

- 대괄호 안에 컬럼명(문자열) 입력
- 결과: 1차원 → Series 타입
- Series의 index = 회사명, value = 해당 컬럼 값

7. 멀티 컬럼 선택

[개념] 대괄호 안에 리스트를 전달하면 여러 컬럼을 선택합니다. 결과는 2차원이므로 DataFrame 타입으로 리턴됩니다.

멀티 컬럼 선택

```
# 멀티 컬럼 선택 -> DataFrame 리턴
```

```
print(df[['PER', 'PBR']])
```

```
# PER PBR
```

```
# NAVER 39.88 4.38
```

```
# 삼성전자 8.52 1.45
```

```
# LG전자 10.03 0.87
```

```
# 카카오 228.38 2.16
```

- 대괄호 안에 반드시 리스트 형태로 전달
- 결과: 2차원 → DataFrame 타입

[주의] [주의] 튜플로 넘기면 오류 발생! df[('PER','PBR')] X -> df[['PER','PBR']] O

8. 로우 선택 (단일 행)

[개념] 행 선택은 반드시 iloc 또는 loc 속성을 사용합니다. 대괄호[]는 컬럼 선택 전용이므로 행 선택에는 사용하지 않습니다.

속성	기준	문법
iloc	행번호 (정수)	df.iloc[1]
loc	인덱스 레이블	df.loc['삼성전자']

단일 행 선택

```
# iloc — 행번호로 선택
```

```
print(df.iloc[0])
```

```
# 종가 157000.0
```

```
# PER 39.88
```

```
# PBR 4.38
```

```
# Name: NAVER, dtype: float64
```

```
# loc — 인덱스 레이블로 선택

print(df.loc['NAVER'])

# 종가 157000.0

# PER 39.88

# PBR 4.38

# Name: NAVER, dtype: float64
```

- 하나의 행 선택 → 1차원 → Series 타입
- 이때 Series의 index = 컬럼명, value = 해당 행의 데이터

9. 멀티 로우 선택

멀티 행 선택

```
# iloc — 행번호 리스트

print(df.iloc[[0, 2]])

# 종가 PER PBR

# NAVER 157000 39.88 4.38

# LG전자 68800 10.03 0.87

# loc — 인덱스 레이블 리스트

print(df.loc[['NAVER', 'LG전자']])

# 종가 PER PBR

# NAVER 157000 39.88 4.38

# LG전자 68800 10.03 0.87
```

- 여러 행 선택 → 2차원 → DataFrame 타입
- iloc: 행번호 리스트 / loc: 인덱스 레이블 리스트

10. 로우 슬라이싱

[핵심 주의] iloc는 끝 미포함, loc는 끝 포함! 이 차이를 꼭 기억하세요.

슬라이싱

```
# iloc 슬라이싱 — 끝 미포함

print(df.iloc[0:2])

# 종가 PER PBR

# NAVER 157000 39.88 4.38

# 삼성전자 51300 8.52 1.45

# < - 0번, 1번만 (2번 LG전자 미포함)


# loc 슬라이싱 — 끝 포함

print(df.loc["NAVER":"삼성전자"])

# 종가 PER PBR

# NAVER 157000 39.88 4.38

# 삼성전자 51300 8.52 1.45

# < - NAVER부터 삼성전자까지 포함


# loc 더 넓은 범위

print(df.loc["NAVER":"LG전자"])

# 종가 PER PBR

# NAVER 157000 39.88 4.38

# 삼성전자 51300 8.52 1.45

# LG전자 68800 10.03 0.87

# < - NAVER부터 LG전자까지 모두 포함
```

방법	끝값 포함	예시	결과
iloc[0:2]	X 미포함	df.iloc[0:2]	0번, 1번만

방법	끝값 포함	예시	결과
loc['A':'B']	O 포함	df.loc['NAVER':'LG전자']	NAVER~LG전자 모두

11. 주요 개념 정리

개념/문법	설명
DataFrame	판다스의 2차원 자료구조. 행 · 열로 구성
행번호/열번호	내부적으로 존재 (숨김). iloc로 접근
df['컬럼명']	단일 컬럼 선택 → Series 리턴
df[['컬럼1','컬럼2']]	멀티 컬럼 선택 → DataFrame 리턴 (리스트 필수!)
df.iloc[n]	행번호로 단일 행 선택 → Series 리턴
df.loc['인덱스']	레이블로 단일 행 선택 → Series 리턴
df.iloc[[0,2]]	행번호 리스트로 멀티 행 선택 → DataFrame
df.loc[['A','B']]	레이블 리스트로 멀티 행 선택 → DataFrame
df.iloc[0:2]	슬라이싱 — 끝 인덱스 미포함
df.loc['A':'B']	슬라이싱 — 끝 인덱스 포함 (iloc와 차이!)

[실습] 실습 팁: 주피터 노트북에서 직접 실행해 보세요. 컬럼 선택은 대괄호, 행 선택은 iloc/loc 속성 — 이 구분을 꼭 기억하세요!

DataFrame 생성 & 인덱싱/슬라이싱 연습문제

총 7문제 | 광명코딩학원

Q1. DataFrame 생성 — 3가지 방법 모두 사용

다음 2차원 데이터를 DataFrame으로 생성하세요. 방법 1(컬럼 단위), 방법 2(로우 리스트), 방법 3(로우 딕셔너리) 3가지를 모두 작성하세요.

암호화폐	시가	고가	저가	종가
비트코인	980	990	920	930
리플	200	300	180	180
이더리움	300	500	300	400

힌트: 방법1: dict {컬럼명:[값...]} / 방법2: 2차원 리스트 + columns / 방법3: [{컬럼명:값}, ...]

내 답안:

[공통 데이터] Q2~Q7은 아래 DataFrame을 기준으로 풀어보세요.

종목	종가	PER	PBR
NAVER	157000	39.88	4.38
삼성전자	51300	8.52	1.45
LG전자	68800	10.03	0.87
카카오	140000	228.38	2.16

Q2. 컬럼 선택 — 단일 컬럼

종가 컬럼만 선택하여 출력하세요. 결과 타입은 무엇인가요?

힌트: df['종가'] -> Series 타입

내 답안:

Q3. 멀티 컬럼 선택

PER과 PBR 컬럼을 동시에 선택하여 출력하세요. 결과 타입은 무엇인가요?

힌트: `df[['PER', 'PBR']] -> DataFrame` 타입 (리스트 필수!)

내 답안:

Q4. 로우 선택 — `iloc`와 `loc`

삼성전자 행을 `iloc`와 `loc` 두 가지 방법으로 각각 출력하세요.

힌트: `df.iloc[1]` / `df.loc['삼성전자']`

내 답안:

Q5. 멀티 로우 선택

NAVER와 카카오 행을 동시에 선택하여 출력하세요. `iloc`와 `loc` 두 가지 방법 모두 사용하세요.

힌트: `df.iloc[[0,3]]` / `df.loc[['NAVER','카카오']]`

내 답안:

Q6. 로우 슬라이싱 — `iloc`

`iloc`를 사용하여 NAVER부터 LG전자까지 슬라이싱하세요. 몇 번~몇 번을 사용해야 하나요?

힌트: `df.iloc[0:3]` (끝 미포함 — LG전자는 행번호 2이므로 3까지)

내 답안:

Q7. 로우 슬라이싱 — `loc`

`loc`를 사용하여 삼성전자부터 카카오까지 슬라이싱하세요. `iloc`와 끝값 처리 차이를 설명하세요.

힌트: `df.loc['삼성전자':'카카오']` (끝 포함 — 카카오도 결과에 포함됨)

내 답안:

정답 모음

DataFrame 생성 & 인덱싱/슬라이싱 | 광명코딩학원

Q1. DataFrame 생성 3가지	<pre># 방법1: df=DataFrame({'시가':[980,200,300],'고가':[990,300,500],...}, index=['비트코인','리플','이더리움']) / # 방법2: df=DataFrame([[980,990,920,930],[200,300,180,180],[300,500,300,400]], / # index=['비트코인','리플','이더리움'], columns=['시가','고가','저가','종가']) / # 방법3: df=DataFrame([{'시가':980,'고가':990,'저가':920,'종가':930}, / # {'시가':200,'고가':300,'저가':180,'종가':180},...], index=[...])</pre>
Q2. 단일 컬럼 선택	<pre>print(df['종가']) / # 결과 타입: Series (1차원이므로)</pre>
Q3. 멀티 컬럼 선택	<pre>print(df[['PER', 'PBR']]) / # 결과 타입: DataFrame (2차원이므로) / # 주의: 튜플 X, 반드시 리스트 O</pre>
Q4. 로우 선택 iloc/loc	<pre>print(df.iloc[1]) # 행번호 1 = 삼성전자 / print(df.loc['삼성전자']) # 인덱스 레이블 / # 결과 타입: Series (하나의 행 = 1차원)</pre>
Q5. 멀티 로우 선택	<pre>print(df.iloc[[0, 3]]) # 행번호 0,3 / print(df.loc[['NAVER', '카카오']]) # 레이블 리스트 / # 결과 타입: DataFrame (여러 행 = 2차원)</pre>
Q6. 슬라이싱 iloc	<pre>print(df.iloc[0:3]) / # 0:3 -> 0,1,2번 (3번 미포함) / # NAVER, 삼성전자, LG전자 출력 (카카오 제외)</pre>
Q7. 슬라이싱 loc	<pre>print(df.loc['삼성전자':'카카오']) / # 삼성전자~카카오 모두 포함 (끝값 포함!) / # iloc 차이: iloc[1:3]=삼성전자,LG전자만 / loc는 카카오도 포함</pre>

[완료] 수고하셨습니다! DataFrame 생성 3가지 방법과 컬럼/행 인덱싱, 슬라이싱까지 완료했습니다. iloc는 끝 미포함, loc는 끝 포함 — 이 차이를 꼭 기억하세요!